Function `parseInt` converts a string to an integer.

```
/**
* Convert a string to an integer.
*
* @param String - A string to be parsed.
* @returns an integer parsed from the string
*/
```

For example:

```
var string = "12345678";
var parsedInteger = parseInt(string);
```

parseFloat

Function `parseFloat` converts a string to a floating-point number, which is a number with a decimal point.

```
/**
* Convert a string to a float.
*
* @param String - A string to be parsed.
* @returns a float parsed from the string
*/
```

For example:

```
var string = "12345678978987654321.98765432123456789";
var parsedFloat = parseFloat(string);
```

parseTPAltkey

Function `parseTPAltkey` parses the alternative key of a tracked process or an event to get its properties including the scheme, party, logical system, type, and ID.

```
/**
* Parse the altkey of a tracked process or event.
*
* @param altkey - A string of alternative key.
* @returns Object of the parsed result
*/
parseTPAltkey(altkey);
```

The function returns an object value with the following properties:

- `scheme`: "xri://sap.com/id", a fixed value.
- `party`: the LBN ID of the business partner.
- `system`: the name of the logical system that the tracked process comes from.

- `type`: the business or document type of the tracked process.
- `id`: the tracking ID of the tracked process.

Examples:

Using the function:

```
var object = parseTPAltkey("xri://sap.com/id:LBN#10002424:QM7CLNT910:OUTBOUND_DEL_AUTO:1000004601")
?object;
```

Response:

```
{"scheme":"xri://sap.com/id","logicalSystem":"QM7CLNT910","id":"1000004601","type":"OUTBOUND_DEL_AU
```

Using a response property:

```
var trackingId = object.id;
```

parseLocationAltKey

Function `parseLocationAltKey` parses the alternative key of a location to get its properties including the scheme, party, system, type, and ID.

```
/**
* Parse the altkey of a location.
*
* @param altkey - A string of alternative key.
* @returns Object of the parsed result
*/
parseLocationAltKey(altkey);
```

The function returns an object value with the following properties:

- `scheme`: "xri://sap.com/id", a fixed value.
- `party`: the LBN ID of the business partner.
- `system`: the name of the system that the location comes from.
- `type`: the object type of the location.
- `id`: the location ID.

Examples:

Using the function:

```
var pLocationAltKey = "xri://sap.com/id:LBN#10002424:Q8JCLNT774:Location:ShippingPoint:01";
var altkey = parseLocationAltKey(pLocationAltKey);
```

Response:

```
{"scheme":"xri://sap.com/id","party":"LBN#10002424","system":"Q8JCLNT774","type":"ShippingPoint","i
```

Using a response property:

```
var id = altkey.id;
?"ID:" + id;
```

composeLocationAltKey

Function `composeLocationAltKey` combines the properties including the party, system, type, and ID to get the location alternative key.

```
/**
* Compose the location altkey.
*
* @param party - A string of the business partner’s LBN ID.
* @param system - A string of the name of the system that the location comes from.
* @param type - A string of the location object type.
* @param id - A string of the location ID.
* @returns String of the location alternative key
*/
composeLocationAltKey(party, system, type, id);
```

Examples:

Using the function:

```
var cLocationAltKey = composeLocationAltKey(“LBN#10002424”, "Q8JCLNT774", “ShippingPoint”, “01”);
?"composeLocationAltKey:" + cLocationAltKey;
```

Response:

```
"xri://sap.com/id:LBN#10002424:Q8JCLNT774:Location:ShippingPoint:01"
```

convertUTCToTimeZone

Function `convertUTCToTimeZone` converts the UTC time to a specified time zone.

```
/**
* Convert the UTC time to a specified time zone.
*
* @param timestamp - A timestamp of the UTC time.
* @param timeZoneID - A string used to identify the time zone. There are two types of timeZoneID:
*       1. Time offset: the number of hours or minutes a certain time zone is ahead of or behind UT
*       2. Time zone region: time zones defined by IANA Time Zone Database, usually in “Area/Locati
* If the timeZoneID is invalid, the system returns the timestamp as a string.
*
* @returns A string in the “year-month-day hour:minute:second” format
```

```
 */
 convertUTCToTimeZone(timestamp, timeZoneID);
```

Examples:

Converting the time from a string:

```
 var timestamp = toTimestamp("2023-04-13T05:45:00Z");
 var converted = convertUTCToTimeZone(timestamp, "Asia/Shanghai");
 // converted is “2023-04-13 13:45:00”
 var converted2 = convertUTCToTimeZone(timestamp, "UTC+05:30:00");
 // converted2 is “2023-04-13 11:15:00”
```

Converting the time by using the `actualBusinessTimestamp` of `actualEvent` and `TimeZoneID` of a location:

```
 var location = getLocationByAltKey(theLocationAltKey);
 var converted = convertUTCToTimeZone(actualEvent.actualBusinessTimestamp, location.TimeZoneId);
```

Event to Action External Functions

Event to Action External functions interact with third-party systems in their operations. They include:

`post`

Function `post` triggers the process of sending the email.

```
 /**
 * Send the email.
 *
 * @returns null
 */
 directMailTemp.post();
```

For example:

```
 directMailTemp.post();
```

`sendTrackingRequest`

Function `sendTrackingRequest` defines extra logic or conditions for the tracking request before sending it to the visibility provider.

You can also use this function to send tracking requests to multiple business partners to report events on the same tracked process. To achieve this, you must add the `receiver` parameter to all your tracking requests. The function won't work if you only use this parameter in some of your requests.

The function can also log the payload sent to the tracking APIs:

- when your call to the tracking APIs fails,

- or when you have enabled the Event to Action Log of the model in the Manage Models app.

After the message processing, the logs are displayed on the **Correlation Details** page of the Manage Message Logs app and the Manage Event Logs app. For instructions, see View Correlation Payload and Logs.

```
/**
* Send a tracking request to the visibility provider.
*
* @param isResourceTP - Mandatory. Always set the value to "False". The resource tracked process is
*
* @optional param trackingRequestConfig - An optional configuration object that provides the visibi
*          - The structure of this object should be as follows:
*          {
*              "trackingSettings": {
*                  "trackShipment": [boolean]   // Optional. Default is true. Determines whether to
*                  "trackCarbonEmission": [boolean]   // Optional. Default is false. Determines whet
*              }
*          }
*          - These settings allow you to choose whether to track shipments, or carbon emissions, or
*
* @optional param receiver - The receiver's identifier (including LBN ID, SCAC code, or others).
*          - It must follow the pattern "Type#ID", such as SCAC#MAEU or LBN#10002419.
*          - If "Type#" is not prepended, the system will identify it as an LBN ID.
*          - The value cannot be null.
*/
sendTrackingRequest(isResourceTP, trackingRequestConfig, receiver)
```

Examples:

Using the function without the optional parameters:

```
sendTrackingRequest(true);
```

Using the function with all parameters:

```
var trackingRequestConfig = toObject("{}");
var trackingSettings = toObject("{}");
// enable tracking carbon emission
trackingSettings.trackCarbonEmission = true;
trackingSettings.trackShipment = false;
trackingRequestConfig.trackingSettings = trackingSettings;
var receiver =  "10002419";
sendTrackingRequest(true, trackingRequestConfig, receiver);
```

forwardCurrentTP

Function `forwardCurrentTP` forwards the current tracked process, actual event, and planed event in JSON format to a specified destination through an HTTP request.

Before using this function, you should first go to your subaccount on SAP BTP cockpit and configure a destination. In the navigation area, go to **Connectivity** > **Destinations** and select **Create Destination**. When setting up your destination, note the following:

- **URL**: The host name URL cannot end with a slash "/". The parameter must always start with a slash "/".
- **Authentication**: Select one of the following:
    - Basic authentication

- Client certificate authentification
- Oauth 2.0 client credentials: you can select this authentification methond with or without using mTLS. If you use mTLS to retrieve the token:
    1. Check the box **Use mTLS for token retrieval**
    2. Then upload certificates as described in Use Destination Certificates. As a result, the token will be automatically retrieved and filled in the relevant fields.

You can define the key-value pair in the **Additional Properties** section:

- Enter "**sap.lbn.gtt.header.<key>**" : "**<value>**". The key-value pair will be added to the header of the forward request.
- If your destination is protected by a CSRF token, add the following properties:

Property	Description
sap.lbn.gtt.csrf.header	Mandatory. The header is used to store the value of the CSRF token. For the header, you can provide your own one based on your server's configuration. In most cases, it should be "x-csrf-token".
sap.lbn.gtt.csrf.method	For the method, you can use GET or HEAD. The default is GET.
sap.lbn.gtt.csrf.url	The URL is used to fetch the CSRF token. The default is the main URL that you configured in the destination.

- If you use OAuth 2.0 client credentials, you can define the following additional properties to specify the HTTP request method of calling token service:

Property	Description
sap.lbn.gtt.oauthtoken.method	For you to specify which HTTP request method to use when calling token service URL. You can enter POST or GET. If not, it will use the default HTTP request method: POST.
scope	You can decide on the scope when calling Token Service. No default value is provided.
tokenService.addClientCredentialsInBody	For you to specify if the credentials should be included in the request body when using POST method. You can enter "True" or "False": • True: the credentials will be included in the requst body. It's the default value. If you don't enter anything, it will set to the default. • False: the credentials will be encrypted and added in the request header. When you use GET method, no matter what you enter, the credentials will always be encrypted and added in the request header.

For more on the request body's structure, see Example of forwardCurrentTP Request Body.

```
/**
* Forward a current tracked process to a destination.
*
* @param destinationName - the destination name
* @param destinationInPaaSAccount - For most GTT customers, it's set to false. If true, the destina
* @param withOldTPandPE - If true, forward the previous tracked process and planned event. If false
* @param async - If true, execute the function asynchronously. If false, execute the function synch
* @returns null
```

```
 */
forwardCurrentTP(destinationName,destinationInPaaSAccount,withOldTPandPE, async)
```

For example:

```
forwardCurrentTP("destinationName",false,false,false);
```

forwardCustomPayload

Function `forwardCustomPayload` forwards a custom payload to an external system.

Before using this function, you should first go to your subaccount on SAP BTP cockpit and configure a destination. In the navigation area, go to **Connectivity** > **Destinations** and select **Create Destination**. When setting up your destination, note the following:

- **URL**: The host name URL cannot end with a slash "/". The parameter must always start with a slash "/".
- **Authentication**: Select one of the following:
  - Basic authentication
  - Client certificate authentification
  - Oauth 2.0 client credentials: you can select this authentification methond with or without using mTLS. If you use mTLS to retrieve the token:
    1. Check the box **Use mTLS for token retrieval**
    2. Then upload certificates as described in Use Destination Certificates. As a result, the token will be automatically retrieved and filled in the relevant fields.

  You can define the key-value pair in the **Additional Properties** section:

  - Enter "**sap.lbn.gtt.header.<key>**" : "**<value>**". The key-value pair will be added to the header of the forward request.
  - If your destination is protected by a CSRF token, add the following properties:

Property	Description
sap.lbn.gtt.csrf.header	Mandatory. The header is used to store the value of the CSRF token. For the header, you can provide your own one based on your server's configuration. In most cases, it should be "x-csrf-token".
sap.lbn.gtt.csrf.method	For the method, you can use `GET` or `HEAD`. The default is `GET`.
sap.lbn.gtt.csrf.url	The URL is used to fetch the CSRF token. The default is the main URL that you configured in the destination.

  - If you use OAuth 2.0 client credentials, you can define the following additional properties to specify the HTTP request method of calling token service:

Property	Description
sap.lbn.gtt.oauthtoken.method	For you to specify which HTTP request method to use when calling token service URL. You can enter POST or GET. If not, it will use the default HTTP

Property	Description
	request method: POST.
scope	You can decide on the scope when calling Token Service. No default value is provided.
tokenService.addClientCredentialsInBody	For you to specify if the credentials should be included in the request body when using POST method. You can enter "True" or "False": ▪ True: the credentials will be included in the requst body. It's the default value. If you don't enter anything, it will set to the default. ▪ False: the credentials will be encrypted and added in the request header. When you use GET method, no matter what you enter, the credentials will always be encrypted and added in the request header.

```
/**
* Forward a custom payload to an external system.
*
* @param destinationName - The destination name.
* @param customPayload - Contain a user-defined key-value pair that will be sent to the destinatior
* @optional param path - A path pointing to the interface.
*/
forwardCustomPayload(destinationName, customPayload, path)
```

For example, using the function with the optional parameter:

```
var destinationName = "TO_QW9_170";
var customPayload = toObject("{}");
customPayload.trackingId = "asdasd";
customPayload.lastReportedEvent = actualEvent;
customPayload.myArray = ["ABC", "DEF"];
var path = "/path";
forwardCustomPayload (destinationName, customPayload, path);
```

forwardEventToTM

Function `forwardEventToTM` forwards one or multiple events back to TM in SAP S/4HANA. If you have maintained the mapping in the Visibility Provider Integration tab of your model, you can also forward the attachments provided by visibility providers with the event.

Before using this function, you should first go to your subaccount on SAP BTP cockpit and configure a destination. In the navigation area, go to Connectivity > Destinations and select Create Destination. When setting up your destination, note the following:

- URL: The host name URL cannot end with a slash "/". The parameter must always start with a slash "/".
- Authentication: Select one of the following:
  - Basic authentication
  - Client certificate authentification
  - Oauth 2.0 client credentials: you can select this authentification methond with or without using mTLS. If you use mTLS to retrieve the token:

1. Check the box **Use mTLS for token retrieval**
2. Then upload certificates as described in Use Destination Certificates. As a result, the token will be automatically retrieved and filled in the relevant fields.

You can define the key-value pair in the **Additional Properties** section:

- Enter "**sap.lbn.gtt.header.<key>**" : "**<value>**". The key-value pair will be added to the header of the forward request.
- If your destination is protected by a CSRF token, add the following properties:

Property	Description
sap.lbn.gtt.csrf.header	Mandatory. The header is used to store the value of the CSRF token. For the header, you can provide your own one based on your server's configuration. In most cases, it should be "x-csrf-token".
sap.lbn.gtt.csrf.method	For the method, you can use `GET` or `HEAD`. The default is `GET`.
sap.lbn.gtt.csrf.url	The URL is used to fetch the CSRF token. The default is the main URL that you configured in the destination.

- If you use OAuth 2.0 client credentials, you can define the following additional properties to specify the HTTP request method of calling token service:

Property	Description
sap.lbn.gtt.oauthtoken.method	For you to specify which HTTP request method to use when calling token service URL. You can enter POST or GET. If not, it will use the default HTTP request method: POST.
scope	You can decide on the scope when calling Token Service. No default value is provided.
tokenService.addClientCredentialsInBody	For you to specify if the credentials should be included in the request body when using POST method. You can enter "True" or "False": ◦ True: the credentials will be included in the requst body. It's the default value. If you don't enter anything, it will set to the default. ◦ False: the credentials will be encrypted and added in the request header. When you use GET method, no matter what you enter, the credentials will always be encrypted and added in the request header.

The function can also log the payload sent to TM in SAP S/4HANA when you have enabled the Event to Action Log of the model in the Manage Models app. After the message processing, the logs are displayed on the **Correlation Details** page of the Manage Message Logs app and the Manage Event Logs app. For instructions, see View Correlation Payload and Logs.

Forward a single actual event:

```
/**
* Forward a single actual event back to the TM system.
*
* @optional param messageHeader (creationDateTime, recipientBusinessSystemID, recipientPartyLBNNo,
* @optional param event (eventTypeCode, eventReasonCode, eventReasonText, referenceEventTypeCode, r
```

```
 * @param destinationName - The destination name.
 */
 forwardEventToTM (messageHeader, event, destinationName);
```

Forward multiple actual events:

```
 /**
 * Forward multiple actual events back to the TM system.
 *
 * @optional param messageHeader (creationDateTime, recipientBusinessSystemID, recipientPartyLBNNo,
 * @param eventArray (event1, event2, …) - The event array contains multiple event objects. Each eve
 * @param destinationName - The destination name.
 */
 forwardEventToTM (messageHeader, eventArray, destinationName);
```

`messageHeader` table

Parameter (Field)	Description	Value	Type	Default Logic
`creationDateTime`	The point in time when the event is created.	Use the function `toTimestamp` to get the value, such as `2020-11-03T05:47:25.402Z`.	Timestamp	Use the current UTC time.
`recipientBusinessSystemID`			String (60)	Use the value of the current tracked process' `logicalSystem` field.
`recipientPartyStandardID`			String (13)	Use the value of the current tracked process' `partyId` field. **Note** If the value contains the prefix “LBN#”, the prefix will be removed.
`senderPartyInternalID`			String (32)	Use the value of the current tracked process' `carrierInternal` field.
`senderPartyStandardID`			String (13)	Use the value of the current tracked process' `serviceAgentLbnId` field. **Note** If the value contains the prefix “LBN#”, the prefix will be removed.